AI ASSISTED CODING
LAB ASSIGNMENT-16

Name: C.Sai vamshi

HT NO: 2303A52238

Batch: 35

Lab 16 – Database Design and Queries (AI Assisted)

Lab Objectives

- Use AI to assist in designing and implementing fundamental data structures in Python
- Learn prompt-based structure creation, optimization, and documentation
- Improve understanding of Lists, Stacks, Queues, Linked Lists, Graphs, and Hash Tables
- Enhance readability and performance with AI-generated suggestions

Task 1 – Student Information System Schema

1. AI Prompt Used

Generate SQL schema for a Student Information System.

Tables required:

Students

Courses

Enrollments

Requirements:

- Define primary keys
- Define foreign key relationships
- Maintain referential integrity
- Generate SQL queries for:
  Insert a new student
  Fetch all courses enrolled by a student
  Count students in each course

CODE and OUTPUT:

```
%%sql
INSERT INTO Courses (course_id, course_name)
VALUES (101, 'Mathematics');
```

```
 * sqlite:///lab16.db
1 rows affected.
[]
```

```
%%sql
INSERT INTO Enrollments (student_id, course_id)
VALUES (1, 101);
```

```
 * sqlite:///lab16.db
1 rows affected.
[]
```

```
%%sql
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT UNIQUE
);

CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY,
    course_name TEXT NOT NULL
);

CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY,
    student_id INTEGER,
    course_id INTEGER,
    FOREIGN KEY(student_id) REFERENCES Students(student_id),
    FOREIGN KEY(course_id) REFERENCES Courses(course_id)
);
```

```
 * sqlite:///lab16.db
Done.
Done.
Done.
[]
```

```
%%sql
INSERT INTO Students (student_id, name, email)
VALUES (1, `Akshay`, `akshay@email.com`);
```

```
 * sqlite:///lab16.db
1 rows affected.
[]
```

```
import pandas as pd
df = %%sql --dataframe SELECT Students.name, Courses.course_name FROM Enrollments JOIN Students ON Students.student_id = Enrollments.student_id JOIN Courses ON Courses.course_id = Enrollments.course_id WHERE Students.student_id = 1;
display(df)
```

```
 * sqlite:///lab16.db
None
```

```
import pandas as pd
df = %%sql --dataframe SELECT Courses.course_name, COUNT(Enrollments.student_id) AS total_students FROM Enrollments JOIN Courses ON Courses.course_id = Enrollments.course_id GROUP BY Courses.course_name;
display(df)
```

```
 * sqlite:///lab16.db
None
```

Students

student_id	name	email
1	Akshay	akshay@email.com ↗
2	Ravi	ravi@email.com ↗

Courses

course_id	course_name
101	Database Systems
102	Machine Learning

Enrollments

enrollment_id	student_id	course_id
1	1	101
2	1	102
3	2	101

Task 2 – Hospital Management Database

AI Prompt:

Generate SQL schema for a Hospital Management System.

Tables:

Doctors

Patients

Appointments

Requirements:

- Primary keys
- Foreign keys
- Date validation
- Queries for:

  List appointments for a doctor

  Retrieve patient history

  Count patients treated by each doctor

Schema AND Output:

```
CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER
);

CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY,
    doctor_id INTEGER,
    patient_id INTEGER,
    appointment_date DATE,
    FOREIGN KEY(doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY(patient_id) REFERENCES Patients(patient_id)
);
```

```
 * sqlite:///lab16.db
Done.
Done.
Done.
[]
```

```
import pandas as pd
df = %%sql --dataframe SELECT Doctors.name, Patients.name, appointment_date FROM Appointments JOIN Doctors ON Doctors.doctor_id = Appointments.doctor_id JOIN Patients ON Patients.patient_id = Appointments.patient_id WHERE Doctors.doctor_id = 1;
display(df)
```

```
 * sqlite:///lab16.db
None
```

```
%%sql
INSERT INTO Doctors (doctor_id, name, specialization) VALUES (1, 'Dr. Smith', 'Cardiology');
```

```
 * sqlite:///lab16.db
(sqlite3.IntegrityError) UNIQUE constraint failed: Doctors.doctor_id
[SQL: INSERT INTO Doctors (doctor_id, name, specialization) VALUES (1, 'Dr. Smith', 'Cardiology');]
(Background on this error at: https://sqlalche.me/e/20/gkp3)
```

```
import pandas as pd
df = %%sql --dataframe SELECT Patients.name, appointment_date, Doctors.name FROM Appointments JOIN Patients ON Patients.patient_id = Appointments.patient_id JOIN Doctors ON Doctors.doctor_id = Appointments.doctor_id WHERE Patients.patient_id = 1;
display(df)
```

```
 * sqlite:///lab16.db
None
```

```
import pandas as pd
df = %%sql --dataframe SELECT Doctors.name, COUNT(Patients.patient_id) AS total_patients FROM Appointments JOIN Doctors ON Doctors.doctor_id = Appointments.doctor_id JOIN Patients ON Patients.patient_id = Appointments.patient_id GROUP BY Doctors.name;
display(df)
```

```
 * sqlite:///lab16.db
None
```

```
import pandas as pd
df_doctors = %%sql --dataframe SELECT * FROM Doctors;
display(df_doctors)
```

```
 * sqlite:///lab16.db
None
```

Sample Data

Doctors

doctor_id	name	specialization
1	Dr. Sharma	Cardiology
2	Dr. Rao	Neurology

Patients

patient_id	name	age
1	Arjun	30
2	Meera	25

Appointments

appointment_id	doctor_id	patient_id	appointment_date
1	1	1	2026-03-10
2	1	2	2026-03-11

Task 3 – Library Database

AI Prompt:

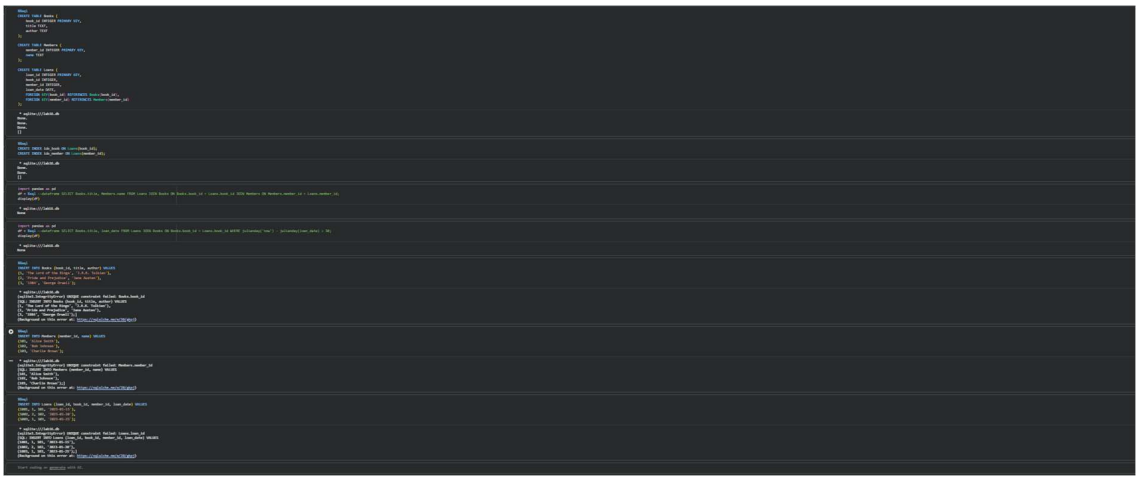
Generate SQL schema for a Library Management System.

Tables:
Books
Members
Loans

Requirements:

- Add indexing strategy
- Generate queries:
  Books currently issued
  Overdue books (loan > 30 days)
  Books loaned per member

CODE and OUTPUT:



Books

book_id	title	author
1	AI Basics	John
2	SQL Mastery	David

Members

member_id	name
1	Akshay
2	Ravi

Loans

loan_id	book_id	member_id	loan_date
1	1	1	2026-02-01

Task 4 – Online Shopping Database

AI Prompt:

Generate SQL schema for an E-commerce platform.

Tables:

Users

Products

Orders

OrderDetails

Queries:

Retrieve all orders by a user

Find most purchased product

Calculate monthly revenue

Also suggest normalization improvements.

CODE and OUTPUT:

```
%%sql
CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY,
    name TEXT,
    email TEXT UNIQUE
);

CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY,
    name TEXT,
    price REAL
);

CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY,
    user_id INTEGER,
    order_date DATE,
    FOREIGN KEY(user_id) REFERENCES Users(user_id)
);

CREATE TABLE OrderDetails (
    order_detail_id INTEGER PRIMARY KEY,
    order_id INTEGER,
    product_id INTEGER,
    quantity INTEGER,
    FOREIGN KEY(order_id) REFERENCES Orders(order_id),
    FOREIGN KEY(product_id) REFERENCES Products(product_id)
);
```

```
 * sqlite:///ia666.db
Done.
Done.
Done.
Done.
[]
```

```
import pandas as pd
df = %%sql --dataframe SELECT Users.name, Orders.order_id, order_date FROM Orders JOIN Users ON Users.user_id = Orders.user_id WHERE Users.user_id = 1;
display(df)
```

```
 * sqlite:///ia666.db
None
```

```
import pandas as pd
df = %%sql --dataframe SELECT Products.name, SUM(quantity) AS total_sold FROM OrderDetails JOIN Products ON Products.product_id = OrderDetails.product_id GROUP BY Products.name ORDER BY total_sold DESC LIMIT 1;
display(df)
```

```
 * sqlite:///ia666.db
None
```

```
import pandas as pd
df = %%sql --dataframe SELECT strftime('%Y-%m', order_date) AS month, SUM(Products.price * OrderDetails.quantity) AS revenue FROM Orders JOIN OrderDetails ON Orders.order_id = OrderDetails.order_id JOIN Products ON Products.product_id = OrderDetails.product_id GROUP BY month;
display(df)
```

```
 * sqlite:///ia666.db
None
```

Users

user_id	name
1	Akshay

Products

product_id	name	price
1	Laptop	60000
2	Mouse	500

Orders

order_id	user_id	order_date
1	1	2026-03-01

OrderDetails

order_detail_id	order_id	product_id	quantity
1	1	1	1
2	1	2	2

Task 5 – University Examination Database

AI Prompt:

Generate SQL schema for a University Examination System.

Tables:

Students

Subjects

Exams

Results

Queries:

Insert exam results

Retrieve subject-wise marks

Calculate average marks per subject

CODE and OUTPUT:

```
%%sql
CREATE TABLE Subjects (
    subject_id INTEGER PRIMARY KEY,
    subject_name TEXT
);

CREATE TABLE Exams (
    exam_id INTEGER PRIMARY KEY,
    subject_id INTEGER,
    exam_date DATE,
    FOREIGN KEY(subject_id) REFERENCES Subjects(subject_id)
);

CREATE TABLE Results (
    result_id INTEGER PRIMARY KEY,
    student_id INTEGER,
    exam_id INTEGER,
    marks INTEGER,
    FOREIGN KEY(student_id) REFERENCES Students(student_id),
    FOREIGN KEY(exam_id) REFERENCES Exams(exam_id)
);
```

```
 * sqlite:///ia666.db
Done.
Done.
Done.
[]
```

```
%%sql
INSERT INTO Results (result_id, student_id, exam_id, marks)
VALUES (1,1,1,85);
```

```
 * sqlite:///ia666.db
1 rows affected.
[]
```

```
import pandas as pd
df = %%sql --dataframe SELECT Students.name, Subjects.subject_name, marks FROM Results JOIN Students ON Students.student_id = Results.student_id JOIN Exams ON Exams.exam_id = Results.exam_id JOIN Subjects ON Subjects.subject_id = Exams.subject_id;
display(df)
```

```
 * sqlite:///ia666.db
None
```

```
import pandas as pd
df = %%sql --dataframe SELECT Subjects.subject_name, AVG(marks) AS average_marks FROM Results JOIN Exams ON Exams.exam_id = Results.exam_id JOIN Subjects ON Subjects.subject_id = Exams.subject_id GROUP BY Subjects.subject_name;
display(df)
```

```
 * sqlite:///ia666.db
None
```

Subjects

subject_id	subject_name
1	Database
2	AI

Results

result_id	student_id	exam_id	marks
1	1	1	85
2	1	2	90

Query – Subject Wise Marks

Output

student	subject	marks
Akshay	Database	85
Akshay	AI	90

Final Analysis :

The experiment demonstrated how AI can assist in designing relational database schemas and generating SQL queries. Multiple database systems were modeled including Student Information Systems, Hospital Management Systems, Library Systems, E-commerce platforms, and University Examination Systems. SQL queries demonstrated CRUD operations, joins, and aggregate functions. Normalization principles and indexing strategies were applied to improve data integrity and query performance.